

Общий TCP Host

Архитектурный анализ, жизненный цикл, подключение новых анализаторов и инструкция по эксплуатации

Параметр	Значение
Решение	SysmexCS2000Ready
Платформа	.NET 9.0 / Windows Service
Рассматриваемый слой	AnalyzerService.Transport
Статус вывода	Архитектура корректна, требуются точечные улучшения
Дата анализа	21 сентября 2026

Документ описывает фактический код решения после выноса TCP-сервера из двух Sysmex-драйверов. Выводы разделяют уже корректные решения, эксплуатационные ограничения и изменения, необходимые для действительно универсального подключения новых приборов.

1. Итоговая оценка

Вынос TCP-host в отдельный проект выполнен по правильной границе ответственности. AnalyzerService.Host управляет конфигурацией и жизненным циклом, AnalyzerService.Transport управляет сокетами, а DLL-драйверы интерпретируют протокол. Объединение Transport с Host создало бы обратную зависимость плагинов от исполняемого приложения и ухудшило бы изоляцию загрузки.

Критерий	Оценка	Комментарий
Разделение ответственности	Хорошо	TCP-код не знает ASTM и Host Online; host не знает детали протокола.
Повторное использование	Хорошо	Оба драйвера ссылаются на один проект и один исходный TcpHost.
Изоляция DLL	Хорошо	Transport загружается вместе с каждой DLL в её collectible AssemblyLoadContext.
Расширяемость протоколов	Требуется правки	Validator разрешает только два известных значения Protocol.
Автоматическое подключение DLL	Частично	Runtime загружает путь из JSON, но package.ps1 публикует драйверы вручную.
Остановка	Работает	CancellationToken + закрытие клиента/listener разблокируют сетевые ожидания.
Отказоустойчивость старта	Требуется правки	Нет явного rollback уже запущенных runtime при ошибке следующего.
Проверка транспорта	Недостаточно	Отдельных loopback-тестов TcpHost пока нет.

1.1. Главная рекомендация

Оставить AnalyzerService.Transport отдельной библиотекой. Следующим этапом убрать из host жёсткий список протоколов, добавить проверку конфликтующих endpoint и покрыть TcpHost интеграционными тестами на loopback. После этого добавление нового драйвера не потребует правок рабочего кода службы.

2. Почему Transport не следует сливать с Host

Вариант	Плюсы	Минусы	Вывод
Отдельный Transport	Переиспользование; тестируемость; нет зависимости от EXE	DLL лежит в каждой папке драйвера	Рекомендуется
TcpHost внутри Host	Одна физическая копия реализации	Драйвер должен ссылаться на EXE или получать host-service через новый ABI	Не рекомендуется сейчас
Factory от Host через Contracts	Host полностью владеет listener; единая политика	Новый контракт и несовместимое изменение Initialize	Возможный v2

Важно различать общий код и общий экземпляр. Сейчас AnalyzerService.Transport является общей кодовой базой, но каждый драйвер получает собственную копию assembly в своём AssemblyLoadContext и собственный TcpHost. Для нескольких приборов это правильная модель: каждый listener имеет отдельный IP/порт и отдельный жизненный цикл.

2.1. Когда factory от Host действительно нужна

- нужна централизованная квота соединений, TLS, единая авторизация или общий мониторинг сокетов;
- драйверы не должны напрямую владеть TcpClient по требованиям безопасности;
- требуется горячее обновление транспортной реализации без перепубликации папок драйверов.

Тогда в Contracts следует добавить IAnalyzerHostServices или ITcpHostFactory и передавать его в Initialize. Это отдельная версия plugin API; перенос конкретного TcpHost в проект Host без такого контракта создаёт неправильное направление зависимостей.

3. Карта проектов и зависимостей

```
AnalyzerService.Host.exe
-> AnalyzerService.Contracts.dll
-> JSON / validation / manager / logging / plugin loading

driver folder A                                driver folder B
SysmexCS2000.Driver.dll                      SysmexCS2000.HostOnline.Driver.dll
AnalyzerService.Transport.dll                 AnalyzerService.Transport.dll
AnalyzerService.Lis.dll                      AnalyzerService.Lis.dll
\                                             /
-> shared AnalyzerService.Contracts.dll in default context
```

Проект	Ответственность	Что не должно находиться внутри
AnalyzerService.Contracts	Стабильные типы IAnalyzerDriver, IAnalyzerLogger, AnalyzerSettings	Сокеты, SQL, конкретный протокол
AnalyzerService.Host	Windows Service, JSON, validation, lifecycle, loading, logging	ASTM/Host Online, парсинг кадров
AnalyzerService.Transport	TcpListener, TcpClient, Start/Accept/Stop	Запросы заказов, STX/ETX, ASTM ACK/NAK
AnalyzerService.Lis	Доступ к заказам и результатам	Жизненный цикл Windows Service
*.Driver	Кодек, сессия протокола, маршрутизация сообщений	Собственная копия TcpListener

3.1. Как AssemblyLoadContext соединяет части

- DriverLoader вычисляет абсолютный путь DLL и создаёт collectible DriverLoadContext.
- AssemblyDependencyResolver ищет AnalyzerService.Transport и AnalyzerService.Lis рядом с главной DLL драйвера.
- AnalyzerService.Contracts специально возвращается в default context; поэтому IAnalyzerDriver в host и plugin является одним типом.
- Остальные зависимости изолированы внутри контекста драйвера. Два драйвера могут иметь разные версии внутренних библиотек.
- LoadedDriver.Dispose вызывает Dispose драйвера, затем context.Unload. Фактическая выгрузка произойдёт после исчезновения всех ссылок и прохода GC.

4. Полная последовательность запуска

```
SCM / console
| Program: Host.CreateApplicationBuilder + AddWindowsService
v
Worker.ExecuteAsync(stoppingToken)
|-- LoadAll(configs/*.json)
|-- ValidateAndThrow(settings)
|-- if ActiveStatus: AnalyzerManager.Add(settings)
v
AnalyzerManager.StartAllAsync
|-- AnalyzerRuntime.StartAsync for each analyzer
|   |-- DriverLoader.Load(DllPath)
|   |-- driver.Initialize(logger, settings)
|   |-- driver.RunAsync(linked token)
`-- Task.WhenAll(all Completion tasks)
```

4.1. Что происходит внутри драйвера

- Initialize проверяет Protocol и создаёт AnalyzerSysmexCS2000 или AnalyzerSysmexCS2000HostOnline.
- Конструктор анализатора создаёт TcpHost, репозиторий ЛИС, кодек/сессию и обработчик результатов.
- RunAsync разбирает IPAddress, синхронно вызывает TcpHost.Start и входит в цикл ожидания.
- AcceptAsync не блокирует поток и возвращает TcpClient после подключения IPU анализатора.
- HandleClientAsync читает поток до отключения, остановки или ошибки. Протокольная логика остаётся в DLL.
- После штатного отключения либо ошибки внешний цикл снова ожидает подключение прибора.

Start и создание простых объектов синхронны. Async применяется только на неопределённо долгих операциях: ожидание клиента, чтение/запись NetworkStream, ожидание завершения runtime и остановка BackgroundService.

5. Что делает общий TcpHost

Метод	Порядок действий	Результат / исключения
constructor	Сохраняет logger и connectionName; проверяет аргументы	Объект без открытого сокета
Start	Под lock проверяет disposed и повторный старт; создаёт TcpListener; Start()	Порт слушается; SocketException при bind
AcceptAsync	Берёт listener; await AcceptTcpClientAsync(token); включает NoDelay; заменяет activeClient	Возвращает TcpClient; cancellation/invalid state
Stop	Под lock отделяет client/listener от состояния; вне lock Dispose/Stop	Акцепт/read разблокируются; повторный вызов допустим
Dispose	Один раз ставит disposed и вызывает Stop	Повторный Dispose безопасен

5.1. Владение ресурсами

TcpHost хранит activeClient, а драйвер одновременно держит ту же ссылку в поле client. Это совместное владение используется только для остановки: оба Dispose идемпотентны. Практически модель работает, но яснее было бы добавить ReleaseClient(TcpClient) либо возвращать объект соединения-владельца, чтобы после завершения сессии TcpHost не хранил уже закрытый объект.

5.2. Потокбезопасность

stateLock защищает listener, activeClient и disposed. Сетевые Dispose/Stop выполняются после выхода из lock, поэтому внутренние системные вызовы не удерживают монитор. Один TcpHost рассчитан на один последовательный клиент. Параллельное обслуживание нескольких IPU одним портом текущей моделью не предусмотрено.

5.3. Замечания к текущему коду

- TcpClient.Connected не является надёжной проверкой живого соединения; корректное обнаружение уже происходит по ReadAsync == 0 или исключению.
- Если Stop одновременно прерывает AcceptTcpClientAsync, возможна SocketException/ObjectDisposedException. Драйвер обычно завершит цикл по связанному token, но полезно нормализовать такую ошибку в OperationCanceledException при запрошенной отмене.
- AcceptAsync заменяет предыдущий activeClient с Dispose. Это соответствует последовательной модели, но должно быть явно закреплено тестом.
- ITcpHost пока не внедряется через конструктор анализатора, поэтому интерфейс почти не повышает тестируемость. Следует принимать ITcpHost либо factory в конструкторе внутреннего analyzer-класса.

6. Как обрабатывается подключение ASTM

```
IPU -> TcpHost.AcceptAsync -> TcpClient/NetworkStream
-> AstmSession.ReceiveMessageAsync
   ENQ / ACK-NAK / frame number / checksum / EOT
-> AstmMessageParser
   Q: repository.GetOrder -> BuildOrder/000/999 -> SendMessageAsync
   R: SysmexResultHandler.Handle (если ResultHandlerStatus=true)
-> disconnect/error -> cleanup -> new AcceptAsync
```

Транспорт знает только TCP. Таймауты, ACK/NAK, кадры и checksum принадлежат AstmSession и Protocol-классам, что подтверждает правильность выделенной границы.

7. Как обрабатывается Sysmex Host Online

```
IPU -> TcpHost.AcceptAsync -> NetworkStream
-> ReadTextAsync: ищет STX, читает до ETX, лимит 255
-> AddBlock: группирует 01..NN по типу и Sample ID
-> HostOnlineCodec
   R221: repository.GetOrder -> S221 (000/999/параметры)
   D121/D221: HostOnlineResultHandler.Handle
-> WriteTextAsync: STX + ASCII body + ETX
```

Чтение STX/ETX и ограничение длины являются частью Host Online presentation layer и правильно остаются в драйвере, а не переносятся в общий TcpHost.

8. Полная последовательность остановки

```
Windows SCM sends Stop
v
Worker.StopAsync(timeout token)
|-- AnalyzerManager.StopAllAsync
|   |-- each AnalyzerRuntime.StopAsync
|   |   |-- stop.Cancel()
|   |   |-- driver.StopAsync(timeout token)
|   |   |   |-- localStop.Cancel()
|   |   |   |-- client.Dispose()
|   |   |   |-- TcpHost.Stop(): client.Dispose + listener.Stop
|   |   |-- await runTask.WaitAsync(timeout token)
|   |-- base.StopAsync
v
DI disposes AnalyzerManager -> Runtime -> LoadedDriver
-> driver.Dispose -> TcpHost.Dispose -> AssemblyLoadContext.Unload
```

8.1. Почему нужны и token, и закрытие сокета

CancellationToken сообщает async-операциям о штатной отмене. Закрытие TcpClient и TcpListener является дополнительной гарантией разблокировки активного чтения и ожидания accept. После выхода RunAsync runtime ждёт задачу, затем DI освобождает драйвер и контекст загрузки.

8.2. Риски остановки

- StopAllAsync останавливает анализаторы последовательно. Медленный первый драйвер может израсходовать общий timeout и не оставить времени остальным.
- Если StopAsync драйвера вернёт завершённую задачу, но RunAsync не завершится, WaitAsync корректно ограничит ожидание токеном SCM.
- Dispose вызывается позже контейнером. При аварии Initialize текущий AnalyzerRuntime не освобождает уже загруженный LoadedDriver — это следует исправить через try/catch rollback.

9. Выявленные улучшения по приоритетам

Приоритет	Проблема	Рекомендуемое изменение
P1	Validator содержит whitelist двух Sysmex Protocol	Проверять непустое Protocol; совместимость оставлять конкретной DLL.
P1	Ошибка старта одного runtime не откатывает предыдущие явно	В StartAllAsync при ошибке отменить/остановить уже запущенные runtime.
P1	Initialize/RunAsync может упасть после Load	В AnalyzerRuntime.StartAsync освободить LoadedDriver и linked CTS в catch.
P1	Не проверяются одинаковые IP/port	До старта сгруппировать активные TCP-конфигурации и отклонить дубликаты.
P2	WorkStatus прочитан, но не влияет на запуск	Определить семантику и применять ActiveStatus && WorkStatus либо удалить поле нельзя из-за совместимости.
P2	HostOnline Initialize допускает повторную инициализацию	Добавить проверку analyzer is not null как в ASTM-драйвере.
P2	package.ps1 перечисляет драйверы вручную	Публиковать проекты по manifest либо копировать подготовленные driver packages.
P2	Нет тестов TcpHost	Loopback: start/accept, cancellation, stop during accept, reconnect, duplicate start.
P3	Синхронный File.AppendAllText в сетевом цикле	При высокой нагрузке использовать bounded channel и одного writer.

10. Инструкция: подключение нового TCP-анализатора

10.1. Создание проекта

```
src/NewAnalyzer.Driver/
NewAnalyzer.Driver.csproj
NewAnalyzerDriver.cs      # единственная публичная IAnalyzerDriver
AnalyzerNewDevice.cs      # orchestration
Protocol/                 # codec, parser, session, exceptions
Lis/                      # mapping результата при необходимости

<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><AssemblyName>NewAnalyzer.Driver</AssemblyName></PropertyGroup>
  <ItemGroup>
    <ProjectReference Include="..\AnalyzerService.Contracts\AnalyzerService.Contracts.csproj" Private="false" />
    <ProjectReference Include="..\AnalyzerService.Transport\AnalyzerService.Transport.csproj" />
    <ProjectReference Include="..\AnalyzerService.Lis\AnalyzerService.Lis.csproj" />
  </ItemGroup>
</Project>
```

10.2. Точка входа DLL

```
public sealed class NewAnalyzerDriver : IAnalyzerDriver
{
    private AnalyzerNewDevice? analyzer;

    public void Initialize(IAnalyzerLogger logger, AnalyzerSettings settings)
    {
        ArgumentNullException.ThrowIfNull(logger);
        ArgumentNullException.ThrowIfNull(settings);
        if (analyzer is not null) throw new InvalidOperationException("Already initialized.");
        if (!string.Equals(settings.Protocol, "NEW_PROTOCOL", StringComparison.OrdinalIgnoreCase))
            throw new ArgumentException("Unsupported protocol.", nameof(settings));
        analyzer = new AnalyzerNewDevice(logger, settings);
    }

    public Task RunAsync(Cancellation_token token) =>
        (analyzer ?? throw new InvalidOperationException("Not initialized.")).RunAsync(token);
    public Task StopAsync(Cancellation_token token) => analyzer?.StopAsync(token) ?? Task.CompletedTask;
    public void Dispose() => analyzer?.Dispose();
}
```

В DLL должна быть ровно одна конкретная публичная реализация IAnalyzerDriver, иначе DriverLoader отклонит сборку как неоднозначную.

10.3. Рабочий цикл нового прибора

```
private readonly TcpHost host;
private readonly CancellationTokenSource localStop = new();
private TcpClient? client;

public AnalyzerNewDevice(IAnalyzerLogger logger, AnalyzerSettings settings)
{
    this.settings = settings;
    host = new TcpHost(logger, settings.AnalyzerName);
}

public async Task RunAsync(Cancellation_token serviceToken)
{
    using var linked = CancellationTokenSource.CreateLinkedTokenSource(
        serviceToken, localStop.Token);
    host.Start(IPAddress.Parse(settings.IPAddress!), settings.Port);
    while (!linked.IsCancellationRequested)
    {
        client = await host.AcceptAsync(linked.Token).ConfigureAwait(false);
        await HandleProtocolAsync(client.GetStream(), linked.Token).ConfigureAwait(false);
    }
}

public Task StopAsync(Cancellation_token token)
{
    localStop.Cancel();
    client?.Dispose();
    host.Stop();
    return Task.CompletedTask;
}
```

В production-коде цикл должен перехватывать ошибки одного соединения, логировать их с контекстом, освобождать клиент и возвращаться к AcceptAsync. OperationCanceledException при linked token означает штатную остановку.

11. Конфигурация и установка

11.1. JSON нового анализатора

```
{
  "AnalyzerName": "New analyzer 1",
  "ConnectionType": "TCPIP",
  "IPAddress": "10.129.1.40",
  "Port": 8030,
  "Isdll": true,
  "DllPath": "drivers\\NewAnalyzer\\NewAnalyzer.Driver.dll",
  "ResultsFolder": "Results",
  "ActiveStatus": true,
  "WorkStatus": true,
  "ResultHandlerStatus": true,
  "OutputFolder": "E:\\Services\\Exchange",
  "ConnectionString": "<connection string>",
  "AnalyzerCode": "NEW1",
  "AnalyzerConfigurationCode": "NEW1",
  "Protocol": "NEW_PROTOCOL"
}
```

До исправления AnalyzerSettingsValidator значение NEW_PROTOCOL будет отклонено. Это главная точка, которая сейчас нарушает заявленную независимость host от конкретных протоколов.

11.2. Публикация DLL

- Опубликовать проект под тот же runtime, что и служба: dotnet publish -c Release -r win-x64 --self-contained false.
- Скопировать всю публикацию, а не только главную DLL, в publish/AnalyzerService/drivers/NewAnalyzer.
- AnalyzerService.Contracts.dll может присутствовать рядом, но DriverLoadContext использует общую contracts-сборку host.
- Поместить JSON в publish/AnalyzerService/configs. AnalyzerName и пара IP/port должны быть уникальны.
- Сначала запустить AnalyzerService.Host.exe в консоли, проверить bind, подключение прибора, обмен и логи; затем устанавливать службу.

11.3. Диагностика запуска

Симптом	Проверить
DLL не найдена	DllPath относительно каталога AnalyzerService.Host.exe; наличие главной DLL.
Найдено 0 реализаций	Public class реализует именно общую AnalyzerService.Contracts.IAnalyzerDriver.
Найдено >1 реализаций	Оставить одну concrete implementation в plugin assembly.
Address already in use	Другая конфигурация/процесс слушает ту же пару IP/port.
IP address not valid	IP принадлежит локальному интерфейсу сервера ЛИС.
Прибор не подключается	Маршрут, firewall, IP/port в IPU, роль прибора как TCP client.
Служба падает при старте	Service/Error logs и Windows Event Log; validation и зависимости DLL.

12. Эксплуатация нескольких анализаторов

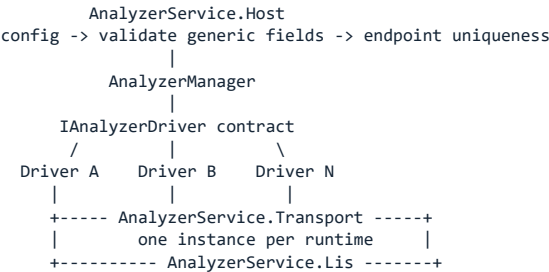
- На каждый прибор создаётся отдельный JSON и отдельный AnalyzerRuntime.
- AnalyzerName используется как ключ manager и каталог логов; дубликаты запрещены.
- Каждый runtime загружает свою DLL и создаёт собственный TcpHost.
- Одинаковый IP допустим, если порты различаются. Одинаковая пара IP/port приведёт к ошибке bind.
- Остановка службы передаёт один сигнал всем runtime, но текущая реализация вызывает StopAsync последовательно.
- Ошибка рабочего цикла одного драйвера завершает Task.WhenAll и считается критической для всей службы; это fail-fast политика.

12.1. Рекомендуемая приёмка

Проверка	Ожидаемый результат
Два прибора, разные порты	Оба listener запускаются и принимают клиентов независимо.
Одинаковый endpoint	Конфигурация отклоняется до частичного старта (после рекомендуемой правки).
Обрыв клиента	Ошибка логируется; драйвер возвращается к accept.

Проверка	Ожидаемый результат
Stop во время accept	RunAsync завершается в пределах timeout без ошибки.
Stop во время read	Сокет закрывается; handler и внешний цикл завершаются.
Неверный кадр	Протокольная ошибка изолирована от TCP-host и имеет код/контекст.
Повторное подключение	Новый клиент принимается после отключения предыдущего.

13. Рекомендуемая целевая архитектура



Plugin package = driver DLL + private dependencies + Transport + Lis
Shared ABI = AnalyzerService.Contracts loaded only by default context

13.1. Минимальный план улучшений

- Сделать AnalyzerSettingsValidator независимым от списка протоколов.
- До загрузки DLL проверить уникальность AnalyzerName и IP/port всех активных конфигураций.
- Добавить rollback в AnalyzerRuntime.StartAsync и AnalyzerManager.StartAllAsync.
- Добавить guard повторной Initialize в Host Online driver.
- Внедрять ITcpHost/factory во внутренние analyzer-классы и добавить loopback-тесты.
- Автоматизировать упаковку новых driver projects или принять готовые plugin-папки как вход package.ps1.

13.2. Окончательный вывод

Рефакторинг TCP-host архитектурно оправдан и лучше исходного дублирования. Перенос реализации в AnalyzerService.Host сейчас ухудшит модульность и создаст зависимость plugin DLL от приложения. Отдельный AnalyzerService.Transport следует сохранить. Для полной расширяемости необходимо убрать знания о конкретных протоколах из validator и packaging, усилить rollback старта и формализовать тесты транспорта.